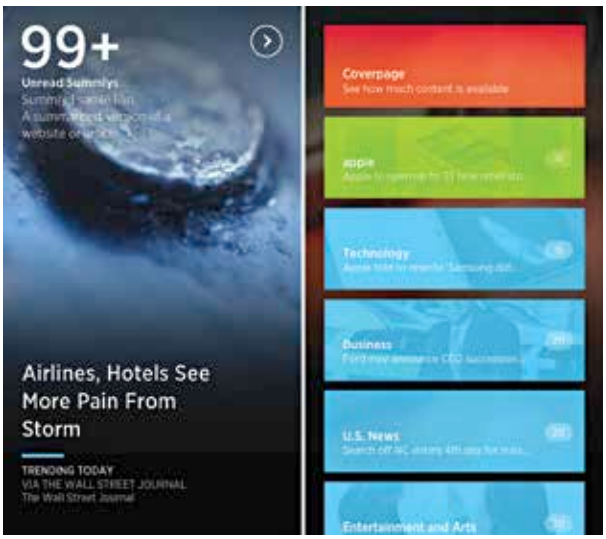
that analyse online reviews on sites like Amazon to gauge the overall response to a product, helping prospective buyers make an informed choice.

Markov models are a class of probabilistic learning algorithms, where the sentence is broken down into its individual constituent words, and represented in the form of a tree or a chain. The system contains a corpus of pre-classified words which are used to perform a preliminary POS classification. However, one word may belong to more than one part of speech, and the classification of such a word can never be completely unambiguous. For e.g. the word 'train' can be used as a noun (a local train) or a verb (to train as a professional athlete). Which of the two is correct in a particular context, is determined by using probabilities that are assigned on the words based on historical data. The words that immediately follow or precede an ambiguous word may also be analysed to assign such probabilities. By repeating this process, the system 'learns' over time; and it's known as Machine Learning.

One excellent application of a Markov chain is in text-prediction, specifically in predictive keyboards in smartphones such as the Android 4.1+ keyboard, or Swiftkey. These employ a similar Markov model, where the next word is predicted based on a probabilistic model, taking into account previously typed pairs of words as well as a corpus of word-pairs found in the training data for accurately predicting the next word that the user might type. Recent updates to Swiftkey have even allowed for the prediction of whole sentences by extrapolating the same algorithm to groups of words. Swiftkey's predictions tend to be eerily accurate, and the app literally 'gets to know you' with time.

A modification of a Markov chain is called Tree-traversal, where individual tokens (letters in a word, words in a sentence or sentences in a block of text) act as nodes of a tree, and each parent node has a child node for each allowed permutation. As an example, a spell-checker would work by traversing a tree with 26 parent nodes (one for each letter of the alphabet) and upto 26 child nodes, who in turn will have at most 26 children of their own. Each level of the tree represents the next letter in a word (so a four letter word would be four levels long), and the algorithm traverses the nodes as the letters are entered in the text document. The number of levels in the tree would be the size of the biggest word in the language. Every time the user enters a combination of letters that is not a valid path in the tree, the program will flag the word as being incorrectly spelled. Suggesting an alternate spelling is far more difficult though, as there is no way of knowing which letter of the word has been incorrectly entered, and will require traversing each letter in the word-tree, trying to find a word that has a spelling similar to the mistyped word.



Summly provided summarized versions of news stories in a gorgeous interface. No wonder Yahoo bought it!

LexRank and Automatic Summarization

Thanks to the ubiquity of the Web, people are finding themselves at the receiving end of a firehose of new information everyday, from email and social media updates to news stories or feeds from blogs and websites. It is practically impossible to consume and digest all of this information in a sane manner, which has given rise to the need for automatic summarization of texts.

Popular iPhone app Summly provided summaries of top news stories in an elegant, well designed interface until it was acquired by Yahoo, making the 17 year old prodigy behind the app richer by $30 million. Summify, a similar service that aggregated content from all your social feeds and delivered it in via email, web or email was acquired by Twitter last year. Not to be left behind, Google acquired Wavii, a startup that offered a personalized, summarized news feed to users by combining stories from social networks and other web content.

One of the most popular algorithms for performing automatic summarization is the LexRank algorithm which gets its name from Google PageRank that powers its searchengine. It works by using a graph-like structure to represent the sentences and their relationships with each other. Each node of the graph is represented by a sentence, and the edges connecting these nodes indicate the similarity between the sentences. The shorter this edge, (or the closer two nodes are) the more similar in meaning they are. The similarity of the sentences is calculated by measuring the frequency of the words occurring in the sentences. This leads to low frequency words being assigned a higher priority in the summary. The importance of a sentence is determined



Swiftkey learns your typing habits over time, which can get slightly... embarrassing